SBA301 – SLOT 29

CHAPTER 17 – HIBERNATE ARCHITECTURE, CONFIGURATION, MAPPING

PART B: MANY-TO-MANY DEMO • DAO • REPOSITORY • SERVICE

CORE: Tư duy trung tâm của Slot 29: Relationship mapping chỉ là một phần của persistence design. Sinh viên phải hiểu đồng thời ba việc: (1) object graph Student ↔ Book được ánh xạ thành join table như thế nào; (2) Session/Transaction được quản lý ở đâu; và (3) vì sao DAO, Repository và Service phải có trách nhiệm khác nhau. Mục tiêu không phải chỉ 'CRUD chạy', mà là tạo một persistence architecture có thể mở rộng, test và bảo trì.

Thông tin	Giá trị
Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Slot	29
Chủ đề	Chapter 17 Part B – Many-to-Many demo, DAO, Repository, Service
CLO chính	CLO5 – Apply Hibernate persistence and layered architecture
Ngữ liệu xuyên suốt	Library Management – Student ↔ Book – Lab 07 bridge
Course baseline	JDK 21 • Maven • Hibernate course track • SQL Server • IntelliJ
Thời lượng gợi ý	135 phút trên lớp + bài tập/AI verification ngoài giờ
Learning flow	Prepare → Model → Map → Implement → Layer → Break → Diagnose → Verify → Transfer

NOTE: Scope lock: Slot 29 đi sâu Many-to-Many và layering DAO → Repository → Service. Không đi sâu JavaFX screen/event handling; UI chỉ là consumer ở cuối flow. Không mở rộng sang Spring Data repository abstraction; current Hibernate Data Repositories chỉ được đặt ở production/current-ecosystem radar.

CÁCH SỬ DỤNG GIÁO TRÌNH

Phần IV là knowledge base lý thuyết mở rộng. Phần VI là implementation workbook từng bước. Trong mọi bước, sinh viên phải kiểm object graph, generated SQL và join-table state; không kết luận mapping đúng chỉ vì application không throw exception.

Pha	Sinh viên làm gì?	Evidence
Prepare	Chạy baseline Slot 28, LibraryDB và SessionFactory.	Baseline
Model	Vẽ Student–Book cardinality + join table.	ER/object map
Map	Cài @ManyToMany, owning/inverse side.	Entity source
Implement	Viết DAO CRUD/association operations.	DAO evidence
Layer	Bọc DAO qua Repository rồi Service.	Call-chain trace
Break	Sai mappedBy/cascade/session/transaction.	Failure evidence
Diagnose	Đọc mapping exception, SQL, join rows.	Debug note
Verify	Persist/update/delete và query qua layers.	DB/test matrix
Transfer	Đánh giá khi nào Many-to-Many nên thay bằng association entity.	Design worksheet

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC SLOT

0.1. Kiến thức cần ôn lại

- Slot 28: SessionFactory, Session, persistence context, lifecycle, transaction, HQL.
- @Entity, @Id, @GeneratedValue, @Column, @ManyToMany và @JoinTable ở mức cơ bản.
- SQL Server: PK/FK, composite key, join table, transaction.
- Java Collection: Set/List, equals/hashCode awareness.
- 3-layer concept: data access, repository abstraction, business/service.
- Maven project structure và Hibernate configuration.

0.2. Công cụ và baseline

Thành phần	Yêu cầu	Kiểm tra
JDK	21	java -version
Maven	3.6.3+ / wrapper	mvn -v
IDE	IntelliJ IDEA	idea .
DB	LibraryDB	SSMS SELECT DB_NAME()
Hibernate	SessionFactory bootstrap được	smoke run
Entities	Student + Book compile	mvn test/compile
Git	Checkpoint sạch	git status

0.3. Pre-class 3–2–1

3 điều đã biết	2 điều còn mơ hồ	1 câu hỏi muốn kiểm chứng
Many-to-many cần join table; Session quản lý entity; Service chứa use-case.	Owning side thực sự có ý nghĩa gì? Repository khác DAO ra sao?	Nếu chỉ thêm Book vào inverse collection mà không sửa owning side thì join table có INSERT không?

0.4. Baseline checkpoint

```
mvn -q -DskipTests compile
mvn test

# Run project / smoke main
mvn exec:java
# hoặc chạy Main từ IntelliJ
```

- LibraryDB kết nối được.
- SessionFactory build thành công.
- STUDENTS và BOOKS tồn tại hoặc được Hibernate tạo.
- Có ít nhất một Student/Book baseline để test.
- Commit/tag gợi ý: s29-before-many-to-many-layering.

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

1.1. Từ mapping đơn entity đến association + layered persistence

Slot 28 đã có	Slot 29 bổ sung	Câu hỏi trọng tâm
Student/Book entity	Many-to-Many mapping	Join table được ai quản lý?
Session/Transaction	DAO	Ai trực tiếp gọi Hibernate API?
HQL	Repository	Ai định nghĩa persistence contract?
Persistence context	Service	Business use-case/transaction orchestration nằm đâu?
SQL verification	Layered call chain	UI có nên gọi Session trực tiếp không?

1.2. Mục tiêu học tập

1. Mô hình hóa Student–Book many-to-many bằng object graph và relational join table.
2. Giải thích owning side, inverse side, `mappedBy`, `@JoinTable`, `@JoinColumn`.
3. Cài đặt helper methods để đồng bộ hai phía association.
4. Phân tích cascade phù hợp và lý do tránh CascadeType.REMOVE trên shared many-to-many.
5. Thiết kế DAO trực tiếp dùng Session/Transaction/HQL.
6. Phân biệt DAO với Repository về mức abstraction và contract.
7. Thiết kế Service gọi Repository thay vì DAO/UI gọi persistence trực tiếp.
8. Trace use-case từ UI/Main → Service → Repository → DAO → Session → SQL Server.
9. Kiểm chứng join table khi add/remove association.
10. Nhận biết khi many-to-many có thuộc tính riêng và nên refactor thành association entity.

1.3. Mental model

```
Object graph
Student ---------------------- Book
  Set<Book> books      Set<Student> students
       |                      ^
       | owning side?         | inverse side?
       +---------- mapping --+
                     |
                     v
          STUDENTS_BOOKS
         student_id | book_id
                     |
                     v
DAO -> Hibernate Session/Transaction
```

```
 |
Repository -> persistence contract
 |
Service -> business use case / orchestration
 |
UI / Main / Controller consumer
```

1.4. Definition of Done

Chưa đủ	Course-ready
Có @ManyToMany	Owning/inverse side rõ + join table verified
CRUD DAO chạy	Transaction/session safe + exception handling
Repository chỉ copy tên DAO	Repository contract có ý nghĩa abstraction
Service chỉ forward mọi method	Service chứa use-case/business validation/orchestration phù hợp
Add book rồi thấy Java Set đổi	Join table SQL/row thay đổi đúng
Delete Student chạy	Không vô tình delete shared Books

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

2.1. Many-to-Many relational model

Một Student có nhiều Book; một Book có thể thuộc nhiều Student. Relational DB biểu diễn bằng join table có hai FK.

2.2. Join table

`STUDENTS_BOOKS(student_id, book_id)` thường có composite PK/unique pair để tránh duplicate association.

2.3. Owning side

Phía khai báo `@JoinTable` là owner trong JPA/Hibernate mapping. Thay đổi collection phía owner quyết định association update.

2.4. Inverse side

Phía `mappedBy` phản chiếu association. Chỉ sửa inverse side có thể không sinh join-table mutation như mong muốn.

2.5. Bidirectional helper methods

`addBook`/`removeBook` nên cập nhật cả `student.books` và `book.students` để object graph nhất quán trong memory.

2.6. Set vs List

Set phù hợp tránh duplicate object association; nhưng equals/hashCode của entity phải được thiết kế cẩn trọng.

2.7. Cascade

Cascade PERSIST/MERGE có thể hợp lý tùy ownership. Cascade REMOVE thường nguy hiểm khi Book được share giữa Students.

2.8. Fetching

Many-to-many collection thường lazy; truy cập ngoài Session có thể gây LazyInitializationException.

2.9. Association entity

Nếu join table có borrowDate/status/quantity/returnedAt thì many-to-many thuần không đủ; map StudentBook/Borrowing entity.

2.10. DAO responsibility

DAO biết Hibernate Session/HQL/mapping. Nó thực hiện low-level persistence operations.

2.11. Repository responsibility

Repository biểu diễn collection-like/domain persistence contract; che giấu DAO/ORM details khỏi Service.

2.12. Service responsibility

Service tổ chức use-case, business validation và transaction boundary phù hợp; không chứa SQL/HQL chi tiết.

2.13. Layer dependency direction

UI/Main → Service → Repository → DAO → ORM/DB. Không đảo ngược và không cho UI gọi Session trực tiếp.

2.14. Transaction boundary

Một use-case có nhiều persistence steps phải atomic. Transaction không nên bị chia vụn chỉ vì mỗi DAO method tự commit độc lập.

2.15. Unit of Work

Hibernate Session tự là unit-of-work; DAO/Repository layer phải tôn trọng lifecycle thay vì mở Session tùy tiện cho từng line.

2.16. HQL in DAO

HQL query thuộc data-access concern; Service chỉ gọi semantic operation như `findByEmail` hoặc `findStudentsByBook`.

2.17. Exception translation awareness

DAO có Hibernate exceptions; upper layers có thể cần translate thành persistence/domain exception để không leak framework.

2.18. Repository interface

Interface có ích khi tạo contract/testing/substitution, nhưng không nên tạo chỉ để tăng số file mà không có abstraction value.

2.19. Service interface

Tương tự repository interface: hữu ích cho contract/DI/testing, nhưng cần tránh interface-for-every-class máy móc nếu project nhỏ.

2.20. Delete semantics

Xóa association khác xóa entity. Remove Book khỏi Student phải delete join row, không nhất thiết delete BOOKS row.

2.21. DAO vs Repository vs Service

Layer	Biết gì?	Không nên biết gì?	Ví dụ
DAO	Session, HQL, Hibernate API	UI workflow	save/update/find entity
Repository	Domain persistence contract	Hibernate implementation details	findStudentByEmail
Service	Use-case/business rule	SQL/HQL/Session details	borrowBook(studentId,bookId)
UI/Main	Input/output interaction	DB/session	call service

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC CỦA BÀI HỌC TRONG SẢN PHẨM THỰC TẾ

3.1. Library domain

Use case	Mapping action	Layer responsibility
Create Student	INSERT STUDENTS	Service → Repository → DAO
Create Book	INSERT BOOKS	Service/Book repository
Assign Book to Student	INSERT STUDENTS_BOOKS	Service orchestrates association
Remove assignment	DELETE join row	Service business rule + DAO persistence
Delete Student	DELETE Student + association rows	Cascade/DB FK semantics
Search Students by Book	HQL join	Repository semantic query

3.2. Why Service matters

Use-case `borrowBook(studentId, bookId)` không nên là một raw DAO CRUD call. Service có thể kiểm Student tồn tại, Book tồn tại, association chưa tồn tại, giới hạn số sách, rồi thực hiện association trong một transaction.

```
borrowBook(studentId, bookId)
  -> load Student
  -> load Book
  -> validate business rules
```

```
-> student.addBook(book)
-> persist association
-> commit
```

3.3. Production transfer

Course design	Production direction
Raw @ManyToMany Student–Book	Association entity Borrowing khi có metadata nghiệp vụ
DAO manually opens Session	Framework-managed transaction/session
Repository implemented over DAO	Spring Data/Jakarta Data/Hibernate repositories where appropriate
Manual DTO/entity usage	Explicit boundary mapping for UI/API
Simple HQL	Fetch plans, projections, pagination, performance tests

IV. NỘI DUNG CHI TIẾT – KNOWLEDGE BASE NỀN TẢNG MANY-TO-MANY VÀ LAYERED PERSISTENCE

READING GUIDE: Phần IV là theory/reference, không phải lab checklist.

4.1. Cardinality và relational representation

- Many-to-many không thể đặt một FK duy nhất trong một trong hai table mà vẫn biểu diễn nhiều-nhiều.
- Join table lưu từng association pair.

4.2. Composite key / uniqueness của join table

- Pair `(student_id, book_id)` nên unique/composite primary key.
- Duplicate association trong object collection nên được ngăn ở cả domain và DB layer.

4.3. Unidirectional vs bidirectional mapping

- Unidirectional chỉ một entity có collection; đơn giản hơn nếu domain không cần navigate chiều ngược.
- Bidirectional cho navigate cả hai chiều nhưng cần consistency helper methods và mappedBy.

4.4. Owning side semantics

- Owner là side Hibernate dùng để write association relation.
- Owner không nhất thiết đồng nghĩa 'business owner' trong domain.

4.5. `mappedBy`

- `mappedBy` trỏ đến Java property ở owner, không phải DB column.
- Sai tên property có thể gây bootstrap/mapping exception.

4.6. `@JoinTable`

- Định nghĩa join table name, joinColumns cho owner FK và inverseJoinColumns cho target FK.
- Column names phải khớp schema hoặc schema-generation strategy.

4.7. Bidirectional consistency

- Nếu chỉ `student.getBooks().add(book)` mà không `book.getStudents().add(student)`, DB có thể đúng nhưng in-memory graph không nhất quán hoặc ngược lại tùy owner.
- Helper method centralizes rule.

4.8. Helper methods

- `addBook(Book)` và `removeBook(Book)` làm association update symmetrical.
- Không expose raw mutable collections nếu domain cần invariant chặt.

4.9. Cascade semantics

- Cascade là propagate entity operation từ parent side qua association.
- `PERSIST` có thể giúp persist new graph; `MERGE` cho detached graph.
- `REMOVE` trên many-to-many shared entities thường không phù hợp.

4.10. Database FK cascade vs ORM cascade

- `CascadeType.REMOVE` khác `ON DELETE CASCADE`.
- ORM operation propagation và database referential action là hai layers khác nhau.

4.11. Lazy loading

- Many-to-many collections nên được coi là potentially lazy.
- Access collection sau Session close có thể fail; fix bằng transaction/fetch plan, không giữ Session mở lâu vô hạn.

4.12. N+1 risk

- Loop Students rồi access books có thể tạo 1 + N queries.
- Giải pháp phụ thuộc use case: join fetch, batch fetching, projection, entity graph.

4.13. Cartesian explosion

- Join fetch nhiều collection cùng lúc có thể nhân số rows.
- Tối ưu ORM phải dựa generated SQL/result cardinality.

4.14. Association entity refactor

- Nếu relation có `borrowDate`, `dueDate`, `status`, `quantity`, `createdBy`, hãy tạo entity Borrowing/StudentBook.
- Khi đó mô hình trở thành two one-to-many/many-to-one relations.

4.15. Entity equality và Set

- HashSet phụ thuộc equals/hashCode.
- Generated IDs làm equality design phức tạp trước persist; cần policy nhất quán.

4.16. DAO pattern

- DAO encapsulates data access implementation details.
- Trong course, DAO là nơi Hibernate Session/HQL được dùng trực tiếp.

4.17. Repository pattern

- Repository mô phỏng collection của aggregate/domain objects.
- Repository method naming nên phản ánh domain intent hơn CRUD mechanics khi hợp lý.

4.18. DAO vs Repository

- DAO thường gần persistence technology/table/entity operations.
- Repository gần domain aggregate/use-case abstraction; trong project nhỏ hai pattern có thể overlap nhưng cần hiểu conceptual difference.

4.19. Service layer

- Service là application/business use-case boundary.
- Nó orchestrates multiple repositories/DAO operations trong transaction và giữ UI không biết persistence details.

4.20. Transaction ownership

- DAO-per-method transaction đơn giản cho CRUD, nhưng use-case multi-step cần transaction ở higher layer.
- Không để một use-case split thành nhiều commits gây partial state.

4.21. Session ownership

- Nếu transaction spans multiple DAO calls, các DAO phải dùng cùng Session/context hoặc framework-managed context.
- Open/close Session inside each DAO call có thể phá unit-of-work.

4.22. Interface boundaries

- `IStudentRepository` và `IStudentService` giúp contract/testing nhưng không tự động làm architecture tốt.
- Interface nên mô tả behavior stable, không mirror implementation boilerplate.

4.23. CRUD DAO
- save: persist; find: get/HQL; update: managed dirty checking/merge; delete: remove.
- API cụ thể phụ thuộc Hibernate version/course baseline.

4.24. Query placement
- HQL đặt ở DAO/repository implementation.
- Service không concatenate HQL hoặc biết table/column.

4.25. Exception handling
- DAO phải rollback khi transaction fail.
- Service có thể translate technical error sang application result/domain exception.

4.26. Resource safety
- Session cần close đúng mọi path.
- try-with-resources giúp giảm leak trong standalone app.

4.27. Testing DAO
- Integration test với SQL Server/DB gần production: persist/reload/query/association.
- Không mock Session rồi kết luận mapping đúng.

4.28. Testing Repository
- Repository contract tests kiểm semantic query/CRUD behavior.
- Implementation detail có thể thay mà service tests ít thay đổi.

4.29. Testing Service
- Service unit test mock repository để kiểm business rule/orchestration.
- Service integration test kiểm transaction + persistence end-to-end.

4.30. UI separation
- JavaFX Controller không mở Hibernate Session trực tiếp.
- UI chỉ gọi service và render result/error.

4.31. Delete association vs delete entity
- Remove Book khỏi Student: xóa join row.
- Delete Book entity: phải xử lý associations/FK trước hoặc theo configured cascade.

4.32. Orphan concept
- Many-to-many thường không dùng orphanRemoval theo cách one-to-many aggregate child.
- Shared entity không phải orphan chỉ vì một parent bỏ association.

4.33. HQL join query
- `select s from Student s join s.books b where b.id = :bookId` query qua object association.
- HQL dùng property names, không join table name trực tiếp trong common association query.

4.34. Fetch join
- `join fetch` có thể load association trong cùng query khi use case cần.
- Không dùng fetch join mặc định cho mọi query; paging + collection fetch có caveats.

4.35. Batch operations
- Association inserts lớn cần transaction size, JDBC batching, flush/clear awareness.
- IDENTITY generation có thể ảnh hưởng batching strategy.

4.36. Concurrency

- Hai transactions cùng sửa collections có thể tạo lost update/logical conflict.
- Production domain có thể dùng optimistic locking @Version hoặc stronger invariant control.

4.37. Repository evolution

- Manual repository giúp học layering.
- Trong ecosystem mới, Jakarta Data/Hibernate Data Repositories cung cấp repository abstraction compile-time/runtime tooling.

4.38. Hibernate Data Repositories 7.4

- Hibernate Data Repositories 7.4 là latest stable đồng phiên bản ORM 7.4 và tương thích Jakarta Data 1.0.
- Đây là production/current ecosystem awareness, không thay course manual Repository implementation.

4.39. Modern HQL

- Hibernate 6/7 HQL mạnh hơn nhiều so với examples cơ bản; JPQL là subset của modern HQL.
- Course query vẫn nên đơn giản, typed, parameterized và verified bằng SQL.

4.40. Current ecosystem 2026

- Hibernate ORM 7.4.13.Final là latest stable release ngày 2026-10-11.
- ORM 8.0 vẫn development/Beta vào thời điểm tài liệu; không dùng làm production default.
- Course giữ version baseline hiện tại; current radar chỉ để sinh viên biết hướng phát triển.

V. KINH NGHIỆM THỰC TIỄN, TIPS VÀ DEBUGGING

Vấn đề	Triệu chứng	Nguyên nhân	Cách chẩn đoán
Join row không tạo	Object Set đổi nhưng DB không đổi	Sửa inverse side/sai owner	SQL log + join table
Duplicate relation	Join table duplicate/error	No Set/unique invariant	collection + constraint
Delete Book xóa quá nhiều	Shared book biến mất	Cascade REMOVE	mapping review
LazyInitializationException	UI access books sau close	Session closed	service/fetch plan
Multiple commits	Use-case partial data	DAO tự transaction từng call	trace transaction
DAO code trong UI	Khó test/bảo trì	layer violation	dependency review
Repository chỉ copy DAO	Abstraction không có giá trị	anemic layering	contract review
Service chứa HQL	Leak persistence concern	layer violation	move query down
N+1	Nhiều SELECT	lazy loop	SQL count
mappedBy error	startup mapping fail	wrong Java property name	exception + source

5.1. Debugging decision tree

```
Association problem
|
+-- Join row missing?
|   +-- Which side owns association?
|   +-- Was owning collection changed?
|   +-- Transaction committed?
|   +-- Cascade needed for new target?
|
+-- Unexpected DELETE?
|   +-- Cascade REMOVE?
|   +-- DB ON DELETE CASCADE?
|   +-- Removing relation vs entity?
|
+-- LazyInitializationException?
|   +-- Session already closed?
|   +-- Use case really needs collection?
|   +-- Fetch join/projection?
|
Layering problem
|
+-- UI imports org.hibernate.Session? -> violation
```

```
+-- Service contains HQL? -> move to DAO/repository
+-- DAO opens independent tx inside multi-step use case? -> transaction boundary review
```

5.2. Tips

- Xác định owner trước khi viết helper method.
- Dùng Set cho shared many-to-many khi duplicate association không có ý nghĩa.
- Không CascadeType.REMOVE qua shared many-to-many nếu chưa chứng minh ownership.
- Một use-case nhiều bước nên có một transaction boundary.
- Service API nên diễn đạt nghiệp vụ: `borrowBook`, không chỉ `saveStudent`.
- Kiểm join table trực tiếp bằng SQL sau association mutation.
- N+1 phải đo bằng SQL log, không đoán từ annotation.
- Khi relation có metadata, refactor association entity sớm.

VI. HƯỚNG DẪN STEP BY STEP THỰC BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP HỆ THỐNG

WORKBOOK RULE: Phần VI là workbook. Mỗi bước: mục tiêu → thao tác → code → chạy → verification → PASS/FAIL.

6.1. Step 1 – Kiểm tra baseline project Slot 28

```
mvn -q -DskipTests compile
mvn test
git status
```

PASS: SessionFactory bootstrap và Student/Book entity compile.

6.2. Step 2 – Mapping Student owning side

```
@Entity
@Table(name = "STUDENTS")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // other fields...

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
        name = "STUDENTS_BOOKS",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "book_id")
    )
    private Set<Book> books = new HashSet<>();

    public void addBook(Book book) {
        books.add(book);
        book.getStudents().add(this);
    }

    public void removeBook(Book book) {
        books.remove(book);
        book.getStudents().remove(this);
    }
}
```

CHECK: Student là owner vì khai báo @JoinTable.

6.3. Step 3 – Mapping Book inverse side

```
@Entity
@Table(name = "BOOKS")
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // isbn, title, author ...

    @ManyToMany(mappedBy = "books", fetch = FetchType.LAZY)
    private Set<Student> students = new HashSet<>();

    public Set<Student> getStudents() {
        return students;
    }
}
```

PASS: `mappedBy = "books"` phải khớp chính xác Java property trong Student.

6.4. Step 4 – Build schema và verify join table

```
SELECT TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_NAME IN ('STUDENTS','BOOKS','STUDENTS_BOOKS');

SELECT *
FROM dbo.STUDENTS_BOOKS;
```

PASS: join table có 2 FK columns đúng mapping.

6.5. Step 5 – Persist Student + Book association

```
try (Session session =
         HibernateUtil.getSessionFactory().openSession()) {

    Transaction tx = session.beginTransaction();

    Student s = new Student(
        "s29@fu.edu.vn", "Slot", "TwentyNine", 8.0);

    Book b1 = new Book("978-001", "Hibernate Basics", "FPT");
    Book b2 = new Book("978-002", "ORM Design", "FPT");

    session.persist(b1);
    session.persist(b2);

    s.addBook(b1);
    s.addBook(b2);

    session.persist(s);

    tx.commit();
}
```

DB verification

```
SELECT * FROM STUDENTS;
SELECT * FROM BOOKS;
SELECT * FROM STUDENTS_BOOKS;
```

PASS: 1 Student + 2 Books + 2 join rows.

6.6. Step 6 – Break-It: chỉ sửa inverse side

```
// Intentional error experiment
book.getStudents().add(student);
// KHÔNG student.getBooks().add(book)
```

Commit rồi kiểm join table. Mục tiêu: chứng minh owner controls association update.

BREAK-IT: Sau experiment phải rollback source về helper method đúng.

6.7. Step 7 – StudentDAO skeleton

```
public class StudentDAO {

    private final SessionFactory sessionFactory;

    public StudentDAO(SessionFactory sessionFactory) {
        this.sessionFactory = sessionFactory;
    }

    public Student save(Student student) {
        try (Session session = sessionFactory.openSession()) {
            Transaction tx = session.beginTransaction();
            try {
                session.persist(student);
                tx.commit();
                return student;
            } catch (RuntimeException ex) {
                tx.rollback();
                throw ex;
            }
        }
    }
}
```

6.8. Step 8 – DAO getAll

```
public List<Student> findAll() {
    try (Session session = sessionFactory.openSession()) {
        return session.createQuery(
            "from Student s order by s.id",
            Student.class
        ).getResultList();
    }
}
```

PASS: returns Students; note `books` may remain lazy.

6.9. Step 9 – DAO findById

```
public Optional<Student> findById(Long id) {
    try (Session session = sessionFactory.openSession()) {
        return Optional.ofNullable(
            session.get(Student.class, id)
        );
    }
}
```

6.10. Step 10 – DAO update

```
public Student update(Student detached) {
    try (Session session = sessionFactory.openSession()) {
        Transaction tx = session.beginTransaction();
        try {
            Student managed = session.merge(detached);
```

```
            tx.commit();
            return managed;
        } catch (RuntimeException ex) {
            tx.rollback();
            throw ex;
        }
    }
}
```

IMPORTANT: Luôn dùng giá trị trả về của `merge()` như managed instance.

6.11. Step 11 – DAO delete

```
public boolean delete(Long id) {
    try (Session session = sessionFactory.openSession()) {
        Transaction tx = session.beginTransaction();
        try {
            Student s = session.get(Student.class, id);
            if (s == null) {
                tx.rollback();
                return false;
            }

            session.remove(s);
            tx.commit();
            return true;
        } catch (RuntimeException ex) {
            tx.rollback();
            throw ex;
        }
    }
}
```

Verification: Student row và join rows bị xử lý; shared Book rows phải còn nếu không cascade REMOVE.

6.12. Step 12 – DAO query student with books

```
public Optional<Student> findByIdWithBooks(Long id) {
    try (Session session = sessionFactory.openSession()) {
        return session.createQuery(
            "select distinct s " +
            "from Student s " +
            "left join fetch s.books " +
            "where s.id = :id",
            Student.class
        )
        .setParameter("id", id)
        .uniqueResultOptional();
    }
}
```

Mục tiêu: tránh accessing lazy collection sau Session close cho use case cần books.

6.13. Step 13 – IStudentRepository

```
public interface IStudentRepository {
    Student save(Student student);
    List<Student> findAll();
    Optional<Student> findById(Long id);
    Optional<Student> findByIdWithBooks(Long id);
    Student update(Student student);
    boolean delete(Long id);
}
```

6.14. Step 14 – StudentRepository implementation

```
public class StudentRepository
        implements IStudentRepository {

    private final StudentDAO dao;

    public StudentRepository(StudentDAO dao) {
        this.dao = dao;
    }

    @Override
    public Student save(Student student) {
        return dao.save(student);
    }

    @Override
    public List<Student> findAll() {
        return dao.findAll();
    }

    @Override
    public Optional<Student> findById(Long id) {
        return dao.findById(id);
    }

    @Override
    public Optional<Student> findByIdWithBooks(Long id) {
        return dao.findByIdWithBooks(id);
    }

    @Override
    public Student update(Student student) {
        return dao.update(student);
    }

    @Override
    public boolean delete(Long id) {
        return dao.delete(id);
    }
}
```

ARCHITECTURE: Course pattern bám slide/Lab. Trong project nhỏ, Repository forwarding 1:1 có thể redundant; mục tiêu ở đây là học abstraction/layer boundaries.

6.15. Step 15 – IStudentService

```
public interface IStudentService {
    List<Student> getStudents();
    Student getStudent(Long id);
    Student createStudent(Student student);
    Student updateStudent(Student student);
    void deleteStudent(Long id);
    void borrowBook(Long studentId, Long bookId);
}
```

6.16. Step 16 – StudentService CRUD

```
public class StudentService
        implements IStudentService {

    private final IStudentRepository students;
```

```
    private final IBookRepository books;

    public StudentService(
            IStudentRepository students,
            IBookRepository books) {
        this.students = students;
        this.books = books;
    }

    @Override
    public List<Student> getStudents() {
        return students.findAll();
    }

    @Override
    public Student getStudent(Long id) {
        return students.findByIdWithBooks(id)
            .orElseThrow(() ->
                new IllegalArgumentException("Student not found"));
    }

    @Override
    public Student createStudent(Student student) {
        return students.save(student);
    }

    @Override
    public Student updateStudent(Student student) {
        return students.update(student);
    }

    @Override
    public void deleteStudent(Long id) {
        if (!students.delete(id)) {
            throw new IllegalArgumentException("Student not found");
        }
    }
}
```

6.17. Step 17 – Service use-case: borrowBook

Nếu Repository/DAO hiện tại mỗi method tự mở transaction, một use-case nhiều bước dễ bị chia transaction. Đối với bài học, trước tiên viết semantic service method; sau đó đánh dấu đây là điểm cần refactor transaction boundary.

```
@Override
public void borrowBook(Long studentId, Long bookId) {
    Student student = students.findByIdWithBooks(studentId)
        .orElseThrow(() -> new IllegalArgumentException("Student not found"));

    Book book = books.findById(bookId)
        .orElseThrow(() -> new IllegalArgumentException("Book not found"));

    boolean alreadyBorrowed = student.getBooks().stream()
        .anyMatch(b -> b.getId().equals(bookId));

    if (alreadyBorrowed) {
        throw new IllegalStateException("Book already associated");
    }

    student.addBook(book);
    students.update(student);
}
```

PRODUCTION: Production/course-extension: tốt hơn khi toàn bộ use-case chạy trong cùng Session/Transaction thay vì load ở tx này rồi merge ở tx khác.

6.18. Step 18 – Main wiring

```
public class Main {
    public static void main(String[] args) {

        SessionFactory sf =
            HibernateUtil.getSessionFactory();

        StudentDAO studentDAO = new StudentDAO(sf);
        BookDAO bookDAO = new BookDAO(sf);

        IStudentRepository studentRepo =
            new StudentRepository(studentDAO);
        IBookRepository bookRepo =
            new BookRepository(bookDAO);

        IStudentService service =
            new StudentService(studentRepo, bookRepo);

        service.getStudents()
            .forEach(System.out::println);

        HibernateUtil.shutdown();
    }
}
```

PASS: Main/UI depends on Service, không import Session để CRUD.

6.19. Step 19 – Layer trace

```
Main / JavaFX Controller
 -> IStudentService.getStudent(id)
 -> StudentService.getStudent(id)
 -> IStudentRepository.findByIdWithBooks(id)
 -> StudentRepository.findByIdWithBooks(id)
 -> StudentDAO.findByIdWithBooks(id)
 -> Session.createQuery(HQL)
 -> Hibernate SQL
 -> SQL Server
 <- Student + books
```

6.20. Step 20 – Remove association safely

```
public void returnBook(Long studentId, Long bookId) {
    Student student = getStudent(studentId);

    Book book = student.getBooks().stream()
        .filter(b -> b.getId().equals(bookId))
        .findFirst()
        .orElseThrow(() ->
            new IllegalStateException("Association not found"));

    student.removeBook(book);
    students.update(student);
}
```

Verification: `DELETE FROM STUDENTS_BOOKS ...`; BOOKS row vẫn còn.

6.21. Step 21 – HQL semantic repository query

```
public List<Student> findStudentsByBook(Long bookId) {
    try (Session session = sessionFactory.openSession()) {
        return session.createQuery(
            "select distinct s " +
            "from Student s join s.books b " +
            "where b.id = :bookId " +
            "order by s.id",
            Student.class
        )
        .setParameter("bookId", bookId)
        .getResultList();
    }
}
```

Repository contract có thể expose `findStudentsByBook`, không expose raw HQL.

6.22. Step 22 – Break-It: CascadeType.REMOVE

Trong branch thử nghiệm, thêm REMOVE vào cascade của Student.books rồi delete một Student có Book dùng chung.

Kiểm DB. Nếu Book bị delete hoặc constraint failure, giải thích vì sao shared many-to-many không có parent-child ownership kiểu aggregate.

BREAK-IT: Khôi phục mapping an toàn sau experiment.

6.23. Step 23 – Break-It: Session closed/lazy

```
Student s = studentDAO.findById(1L).orElseThrow();
// Session inside DAO already closed:
System.out.println(s.getBooks().size());
```

Nếu collection chưa initialized, có thể LazyInitializationException. Fix bằng use-case query/fetch plan, không bằng global EAGER.

6.24. Step 24 – Refactor Many-to-Many thành Borrowing entity

```
@Entity
@Table(name = "BORROWINGS")
public class Borrowing {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(optional = false)
    private Student student;

    @ManyToOne(optional = false)
    private Book book;

    private LocalDate borrowedDate;
    private LocalDate dueDate;

    @Enumerated(EnumType.STRING)
    private BorrowingStatus status;
}
```

Mục tiêu: thấy rõ khi relation có nghiệp vụ, association entity tốt hơn raw @ManyToMany.

6.25. Step 25 – Test matrix

ID	Test	Expected
T01	Persist Student + 2 Books	2 join rows
T02	Add existing Book	1 new join row
T03	Add same Book twice	no duplicate semantic association

ID	Test	Expected
T04	Load with books	collection available after DAO return
T05	Remove association	join row deleted only
T06	Delete Student	Books remain
T07	findStudentsByBook	correct Student set
T08	wrong mappedBy	bootstrap fail
T09	inverse-only mutation	join row not persisted as intended
T10	lazy outside Session	failure or no access per initialized state
T11	service duplicate borrow	business exception
T12	rollback failure	no partial committed state

6.26. Step 26 – Integration Gate G1–G14

Gate	Evidence
G1 Build	compile/test pass
G2 Mapping	owner/inverse clear
G3 Join table	PK/FK + rows verified
G4 Helper	both collections consistent
G5 DAO	CRUD + rollback
G6 Repository	contract hides Hibernate details
G7 Service	use-case boundary
G8 Layer direction	UI/Main → Service → Repository → DAO
G9 Association add	join INSERT
G10 Association remove	join DELETE only
G11 Shared entity safety	Book remains
G12 Query	HQL association query
G13 Break/Fix	mappedBy/cascade/lazy issue diagnosed
G14 Explain	Many-to-Many vs association entity trade-off

6.27. Bài tập tổng hợp hệ thống

1. Hoàn thiện Student–Book bidirectional mapping.
2. Tạo StudentDAO CRUD và query with books.
3. Tạo IStudentRepository + StudentRepository.
4. Tạo IStudentService + StudentService.
5. Tạo Book repository tối thiểu phục vụ borrow use-case.
6. Implement borrowBook/returnBook.
7. Kiểm join table bằng SQL.
8. Chạy negative experiments mappedBy/cascade/lazy.
9. Viết query findStudentsByBook.
10. Refactor prototype Borrowing association entity.
11. Viết architecture note về transaction boundary limitation của manual DAO version.
12. Đề xuất production migration: framework-managed transaction hoặc repository technology hiện đại.

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa	Ví dụ Slot 29
Many-to-Many	Quan hệ nhiều-nhiều	Student ↔ Book
Join Table	Bảng lưu association pairs	STUDENTS_BOOKS
Owning Side	Phía write relationship	Student @JoinTable
Inverse Side	Phía mappedBy	Book.students
mappedBy	Tên Java property ở owner	mappedBy="books"
@JoinTable	Cấu hình join table	STUDENTS_BOOKS
@JoinColumn	Cấu hình FK column	student_id
Cascade	Propagate entity operations	PERSIST/MERGE
Cascade REMOVE	Propagate delete	nguy hiểm với shared Book
LAZY	Deferred association load	books
Fetch Join	Load association trong query	join fetch s.books
Association Entity	Entity đại diện relation có metadata	Borrowing
DAO	Data Access Object	StudentDAO
Repository	Domain persistence abstraction	IStudentRepository
Service	Use-case/business layer	StudentService
Transaction Boundary	Phạm vi atomic unit	borrowBook
Unit of Work	Nhóm changes trong Session	Hibernate Session
Exception Translation	Ẩn framework exception	PersistenceException → app error
Layer Violation	Dependency vượt boundary	UI gọi Session
N+1	N query phụ sau query chính	student.books loop
Bidirectional Association	Navigate 2 chiều	student.books/book.students
Helper Method	Đồng bộ 2 phía	addBook/removeBook

Thuật ngữ	Định nghĩa	Ví dụ Slot 29
Shared Entity	Entity được nhiều owner tham chiếu	Book
Aggregate	Consistency/business boundary	domain concept
Repository Contract	Semantic persistence API	findStudentsByBook

VIII. RESEARCH BASE VÀ AI-BASED LEARNING – 4 NHÓM TRẢI NGHIỆM CUỐI BUỔI

8.1. Quy tắc AI

- Prompt phải dựa source/mapping/project thật; không chấp nhận code giả không compile.
- Relationship claim phải verify bằng generated SQL + join-table state.
- Layering claim phải đối chiếu dependency imports/call chain.
- Current recommendation phải kiểm official Hibernate/Jakarta docs.
- Không gửi credential/database password lên AI.

8.2. NHÓM I – Workflow / execution rule

PROMPT I.1: Trace `student.addBook(book)` đến INSERT vào join table; phân biệt Java helper call với Hibernate flush SQL.

PROMPT I.2: Trace delete Student khi Book được share; xác định SQL và cascade decisions.

PROMPT I.3: Trace `findByIdWithBooks` bằng join fetch từ HQL đến hydrated object graph.

PROMPT I.4: Trace Main → Service → Repository → DAO → Session → DB cho getStudent.

PROMPT I.5: Trace borrowBook và chỉ ra transaction bị tách ở đâu trong manual DAO design.

PROMPT I.6: Trace inverse-only collection mutation và giải thích vì sao owner quyết định join-table write.

PROMPT I.7: Trace LazyInitializationException từ DAO return đến UI access.

PROMPT I.8: Trace merge detached Student có books và chỉ ra managed instance nào được persistence context track.

PROMPT I.9: Trace remove association để chứng minh chỉ join row bị delete.

PROMPT I.10: Vẽ sequence diagram cho một use-case nhiều DAO calls và đề xuất đúng transaction boundary.

8.3. NHÓM II – 10 prompt kỹ thuật nâng cao

PROMPT II.1: So sánh unidirectional và bidirectional @ManyToMany về schema, object consistency, SQL và maintainability.

PROMPT II.2: Phân tích owning side vs inverse side bằng 6 mutation scenarios; dự đoán SQL trước khi chạy.

PROMPT II.3: Phân tích Set/List cho many-to-many, equals/hashCode và duplicate association risks.

PROMPT II.4: Phân tích CascadeType.PERSIST/MERGE/REMOVE/ALL trên shared entities và xây decision matrix.

PROMPT II.5: Phân tích N+1 với Student–Book; so sánh join fetch, batch fetching, subselect fetching, projection.

PROMPT II.6: Phân tích cartesian explosion khi fetch multiple collections và cách đo số row vs số entity.

PROMPT II.7: Phân tích bulk delete association bằng HQL/native SQL và persistence-context staleness.

PROMPT II.8: Thiết kế optimistic-lock strategy khi hai transactions sửa cùng Student.books collection.

PROMPT II.9: Phân tích association entity Borrowing với composite natural key vs surrogate id.

PROMPT II.10: Review DAO/repository code và tìm session leaks, double transactions, stale detached entities, exception handling lỗi.

8.4. NHÓM III – 10 prompt production engineering

PROMPT III.1: Refactor manual DAO transaction design để một Service use-case dùng cùng Session/transaction; đưa 2 phương án standalone và Spring-managed.

PROMPT III.2: Thiết kế repository contract cho Library domain không bị CRUD-centric; nhóm methods theo domain use cases.

PROMPT III.3: Thiết kế Borrowing aggregate cho mượn/trả sách: status, dueDate, renew count, audit, optimistic locking.

PROMPT III.4: Thiết kế integration tests dùng SQL Server container/test DB cho mapping + transaction + join table.

PROMPT III.5: Thiết kế migration scripts cho STUDENTS_BOOKS → BORROWINGS mà không mất dữ liệu.

PROMPT III.6: Thiết kế performance test cho 10k Students × Books; metrics SQL count, latency, memory, DB plan.

PROMPT III.7: Thiết kế caching strategy: Book reference data vs active Borrowing data; second-level cache risks.

PROMPT III.8: Thiết kế audit trail cho borrow/return không dựa vào raw many-to-many.

PROMPT III.9: Thiết kế multi-user concurrency: hai nhân viên cùng mượn một Book copy; tại sao many-to-many model không đủ.

PROMPT III.10: Review architecture JavaFX → Service → Repository → DAO và đề xuất DI/factory/composition root để giảm hard-coded wiring.

8.5. NHÓM IV – 10 prompt current ecosystem / community trends

PROMPT IV.1: Kiểm tra Hibernate ORM latest stable và development versions hiện tại; ghi Java/Jakarta/Spring Boot compatibility.

PROMPT IV.2: Đọc Hibernate ORM 7.4 User Guide về bidirectional many-to-many và so sánh với course mapping.

PROMPT IV.3: Đọc official guidance về @ManyToMany lifecycle/cascade và xác minh việc xóa link rows khi remove association.

PROMPT IV.4: Khảo sát modern Hibernate HQL join/fetch capabilities liên quan collection relationships.

PROMPT IV.5: Khảo sát Hibernate Data Repositories 7.4/Jakarta Data 1.0 và so sánh với manual IStudentRepository trong course.

PROMPT IV.6: Khảo sát Hibernate Processor/type-safe query tooling và giá trị khi refactor repository.

PROMPT IV.7: Kiểm tra ORM 8.0 development status; liệt kê changes WATCH LATER thay vì production adoption.

PROMPT IV.8: Khảo sát association entity best practices trong official Hibernate docs, đặc biệt link entity với extra columns.

PROMPT IV.9: Khảo sát Hibernate statistics/Micrometer/JFR hiện tại để đo N+1 và persistence performance.

PROMPT IV.10: Tạo '2026 Hibernate Layering & Relationship Radar': raw DAO, Repository, Jakarta Data, HQL, association entity, fetch plans, observability, ORM 8.0.

8.6. AI Verification Worksheet

Claim	Nguồn kiểm chứng	Experiment	Kết luận SV
Owner controls relation	source + SQL log	inverse-only mutation	
Cascade REMOVE risky	mapping + DB	delete shared Student	
DAO tx split problem	call trace	multi-step failure	
N+1	SQL count	student.books loop	
Current Hibernate repositories	hibernate.org official	minimal prototype	

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

□ 1. Tôi vẽ được Student–Book join table.

□ 2. Tôi xác định đúng owner và inverse side.

□ 3. Tôi giải thích `mappedBy` dùng Java property name.

□ 4. Tôi viết add/remove helper đồng bộ hai phía.

□ 5. Tôi hiểu cascade REMOVE nguy hiểm với shared entity.

□ 6. Tôi phân biệt delete association và delete entity.

□ 7. Tôi viết StudentDAO CRUD với transaction/rollback.

□ 8. Tôi phân biệt DAO và Repository.

□ 9. Tôi giải thích Service phải chứa use-case/business orchestration.

□ 10. Tôi trace được UI/Main → Service → Repository → DAO → Hibernate → DB.

□ 11. Tôi kiểm join table bằng SQL.

□ 12. Tôi biết cách load association khi Session đã đóng.

□ 13. Tôi nhận biết N+1 và không sửa bằng EAGER everywhere.

□ 14. Tôi biết khi nào nên thay @ManyToMany bằng Borrowing entity.

□ 15. Tôi hiểu limitation transaction của manual DAO-per-method và hướng production.

IX.1. Exit ticket

Câu hỏi	Trả lời
`mappedBy="books"` trỏ tới table/column hay Java property?	

Câu hỏi	Trả lời
Xóa Student có nên xóa Book shared không? Vì sao?	
DAO khác Repository ở abstraction nào?	
Service có nên chứa HQL không?	
Khi join table có dueDate/status, nên refactor thế nào?	

TÀI LIỆU THAM CHIẾU & SOURCE TRACEABILITY

- Course source: Chapter 17 – Demo Hibernate (Many To Many), Create Student/Book, hibernate.cfg.xml, StudentDAO CRUD, IStudentRepository/StudentRepository, IStudentService/StudentService, Main.
- Course source: Lab 07 – Activity 02 Repository Pattern: Maven project, Hibernate config, Student/Book POJO, StudentDAO CRUD, Repository, Service.
- Official Hibernate ORM 7.4 documentation – relationship mapping, runtime, HQL and query guidance.
- Official Hibernate Data Repositories 7.4 – current repository ecosystem/Jakarta Data awareness.

NOTE: Current ecosystem note (verified 2026-10-12): Hibernate ORM 7.4 is latest stable; 7.4.13.Final was released 2026-10-11. Hibernate ORM 8.0 remains development/Beta. Hibernate Data Repositories 7.4 is also latest stable and is compatible with Jakarta Data 1.0. These facts are production/current-radar only; course implementation remains manual DAO → Repository → Service.